

Knock Knock Software Architecture Document

02/02/2025

by Lucas Hasting

Contents

1	Introduction	3
1.1	Definitions	3
1.2	References	3
2	Architectural Representation	4
2.1	Use-Case view	4
2.2	Logical view	5
2.3	Process view	6
2.4	Deployment view	7
2.5	Implementation Views	8
3	Architectural Goals and Constraints	9
3.1	Architectural Goal	9
3.2	Architectural Constraints	9
4	Use-Case View	10
4.1	Use-Case Realizations	10
5	Logical View	10
6	Process View	11
7	Deployment View	12
8	Implementation View	13
8.1	Layers	13
9	Size and Performance	16
10	Quality	16

1 Introduction

The purpose of this document is to lay out the architecture of the Knock Knock application that makes use of the Knock Knock Protocol. In section 2, we look at different views that will be shown of the architecture and how they are used. Section 3 goes over the goal of the architecture along with its constraints. Sections 4 through 8 show the views of the architecture mentioned in section 2. Section 9 describes the characteristics of the architecture that can increase its size and affect performance. Finally, section 10 describes the quality of the architecture, i.e., the non-functional requirements of the architecture.

1.1 Definitions

UML: Unified Modeling Language [2].

Bit: the smallest unit of storage in a digital computer [8].

Megabit: 10^6 bits [7].

Byte: 8 bits [8].

MB: Megabyte (2^{20} bytes) [6].

CPU: Central Processing Unit [11].

CPU Core: a single processing unit [11].

Operating System: a system to manage hardware/software resources [12].

Protocol: a set of rules for transmitting data between computers [9].

jar file: executable compressed collection of java files [13].

Intranet: the local (internal) computer network [10].

Data in transit: data moving across the network [14].

1.2 References

[1]: <https://www.geeksforgeeks.org/use-case-diagram/>

[2]: <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/>

[3]: <https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>

[4]: <https://www.uml-diagrams.org/activity-diagrams-reference.html>

[5]: <https://www.uml-diagrams.org/deployment-diagrams-reference.html>

[6]: <https://www.gbmb.org/mb-to-bytes>

[7]: <https://www.gbmb.org/mbit-to-bit>

[8]: <https://web.stanford.edu/class/cs101/bits-bytes.html>

- [9]: <https://www.britannica.com/technology/protocol-computer-science#:~:text=protocol%2C%20in%20computer%20science%2C%20a,will%20send%20and%20receive%20it.>
- [10]: <https://www.interactsoftware.com/blog/intranet-definition/>
- [11]: <https://www.supermicro.com/en/glossary/core#:~:text=In%20computing%2C%20a%20'Core',tasks%20like%20rendering%203D%20images.>
- [12]: https://en.wikipedia.org/wiki/Operating_system
- [13]: [https://en.wikipedia.org/wiki/JAR_\(file_format\)](https://en.wikipedia.org/wiki/JAR_(file_format))
- [14]: <https://www.digitalguardian.com/blog/data-protection-data-in-transit-vs-data-at-res>
- [15]: <https://www.freecodecamp.org/news/static-variables-in-java/#:~:text=Static%20variables%20and%20static%20methods,an%20instance%20of%20the%20class.>
- [16]: Software Architecture in Practice (Fourth Edition) by Len Bass, Paul Clements, Rick Kazman.

2 Architectural Representation

In the next few subsections, each view's purpose will be described, and the elements of the models used will be defined. The views of the architecture are shown in sections 4 through 8.

2.1 Use-Case view

This view shows how the architecture interacts with other people. For this architecture, we show this using a UML use case diagram. Below, the notation used is shown, along with their definitions:

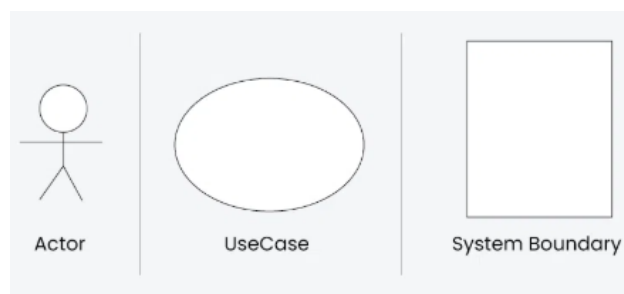


Figure 1: Use Case Diagram [1]

The actor is someone who is using the system, the use case is how the system is used, and the system boundary is the system itself. A line from the actor to a use case represents the use case the actor would perform (a gamer actor would have a use case game, and there would be a line from the gamer actor to the use case game).

2.2 Logical view

This view shows how the architecture is set up as classes, along with their respective jar file. For this architecture, we show this using a UML class diagram. Below, the notation used is shown, along with their definitions.

First the layout of a class is shown below:

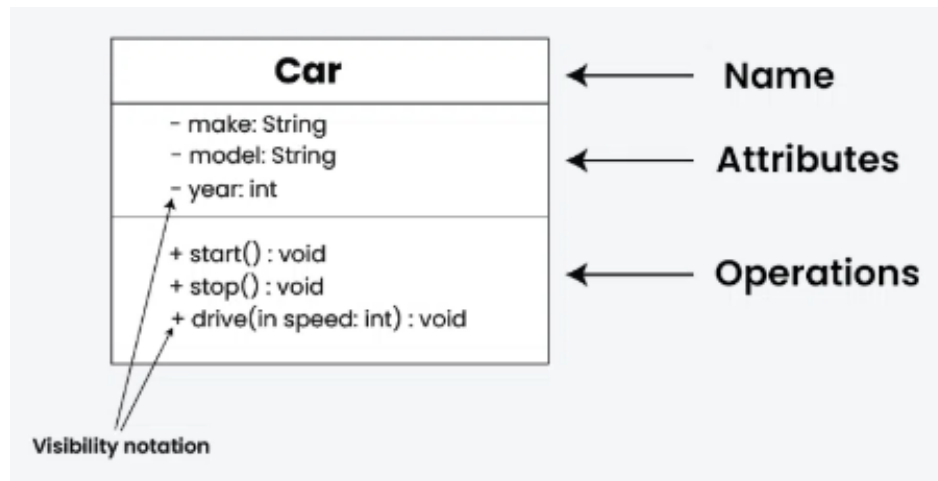


Figure 2: Class Diagram - Class [2]

The attributes of the class are variables that can be used in the class. The + (public) permission allows the attribute to be accessed outside of the class, while the - (private) permission does not. The operations of a class are also referred to as methods [2].

Also, we will represent static attributes and methods by underlining the attributes and/or the methods. A static method/attribute belongs to the class itself instead of an instance of the class [15].

Next, a composition relationship is shown below:

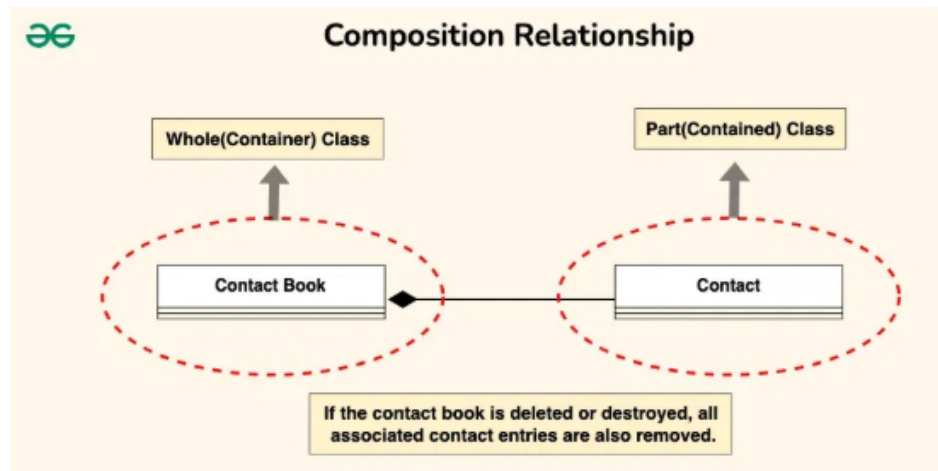


Figure 3: Class Diagram - Composition [2]

Finally, a dependency relationship is shown:

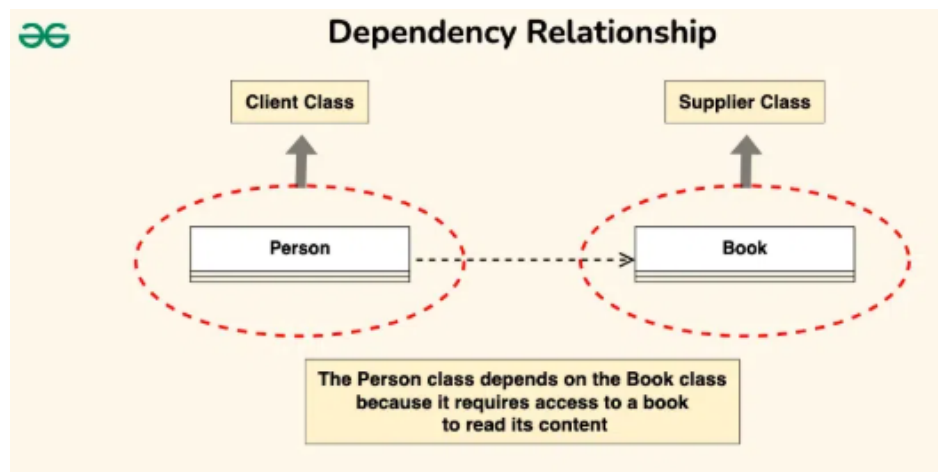


Figure 4: Class Diagram - Dependency [2]

2.3 Process view

This view shows how the architecture is broken down into processes, and how those processes communicate through each step of the architecture. For this architecture, we show this using a UML activity diagram. Below, the notation used is shown, along with their definitions:

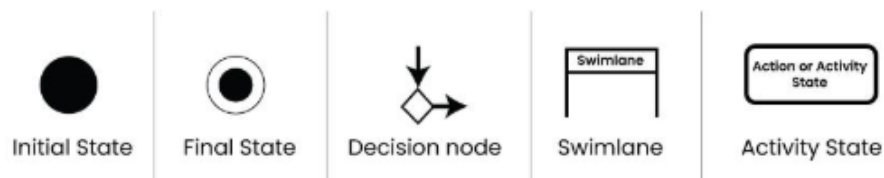


Figure 5: Activity Diagram [3]

The initial state is where the diagram begins, and the final state is when it ends. A swim lane represents the individual process and the activity state describes what the process is currently doing. When a decision is made it will branch into 2 possibilities (yes vs. no). An arrow is used to denote state transitions (going from one state to the next) [3]. We will also use “Event Actions” to represent the transfer of data from one process to the other over the network, this action is shown and described below:

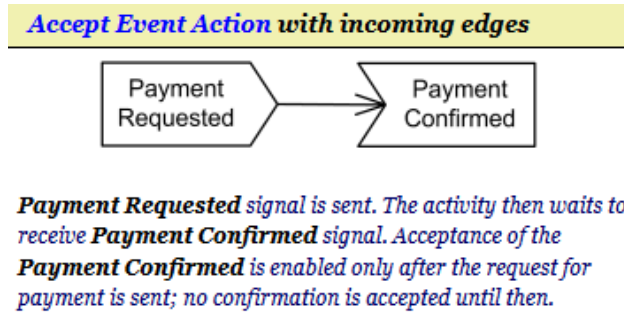


Figure 6: Activity Diagram - Event Action [4]

2.4 Deployment view

This view shows how the architecture is deployed to devices and how they are used. For this architecture, we show this using a UML deployment diagram. Below, the notation used is shown, along with their definitions:

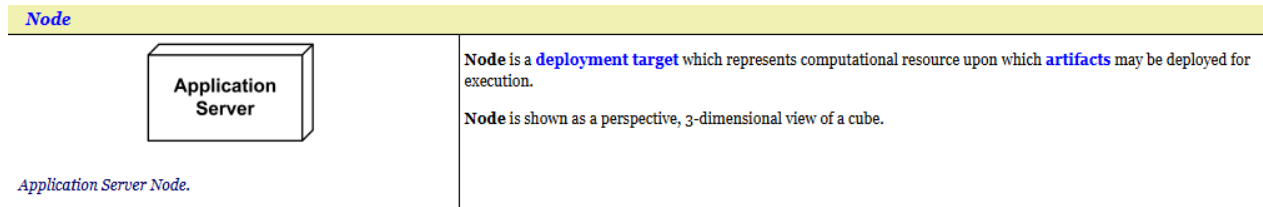


Figure 7: Deployment Diagram - Node [5]

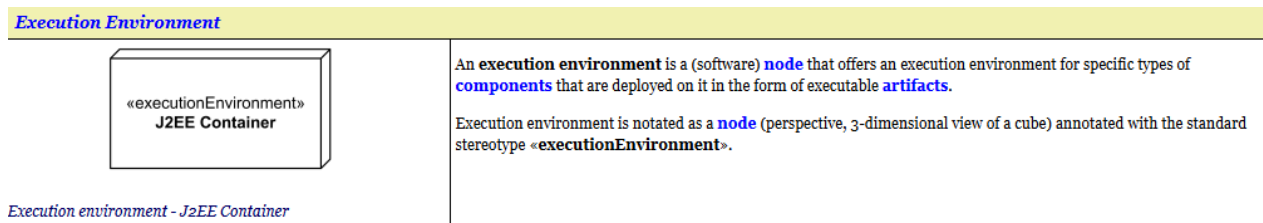


Figure 8: Deployment Diagram - Execution Environment [5]

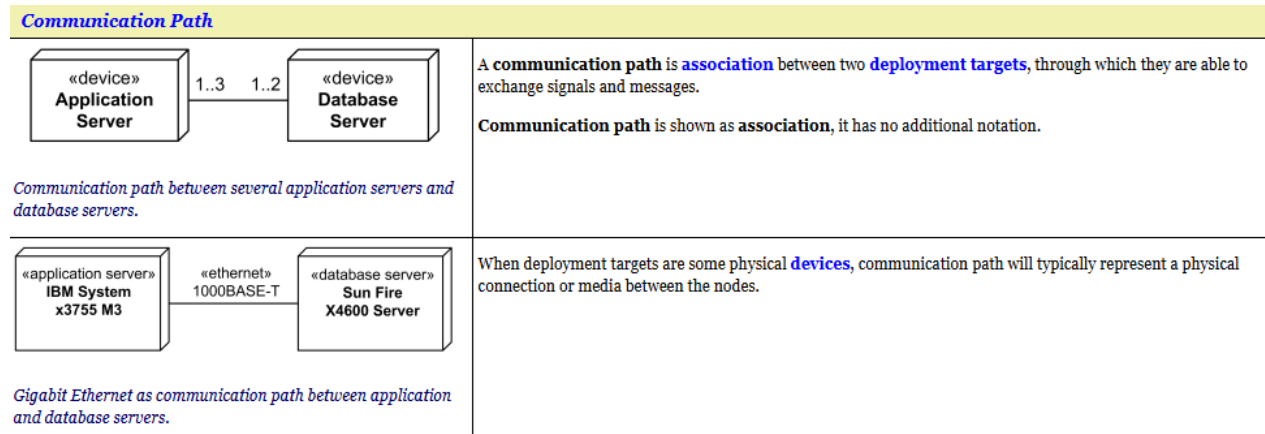


Figure 9: Deployment Diagram - Communication Path [5]

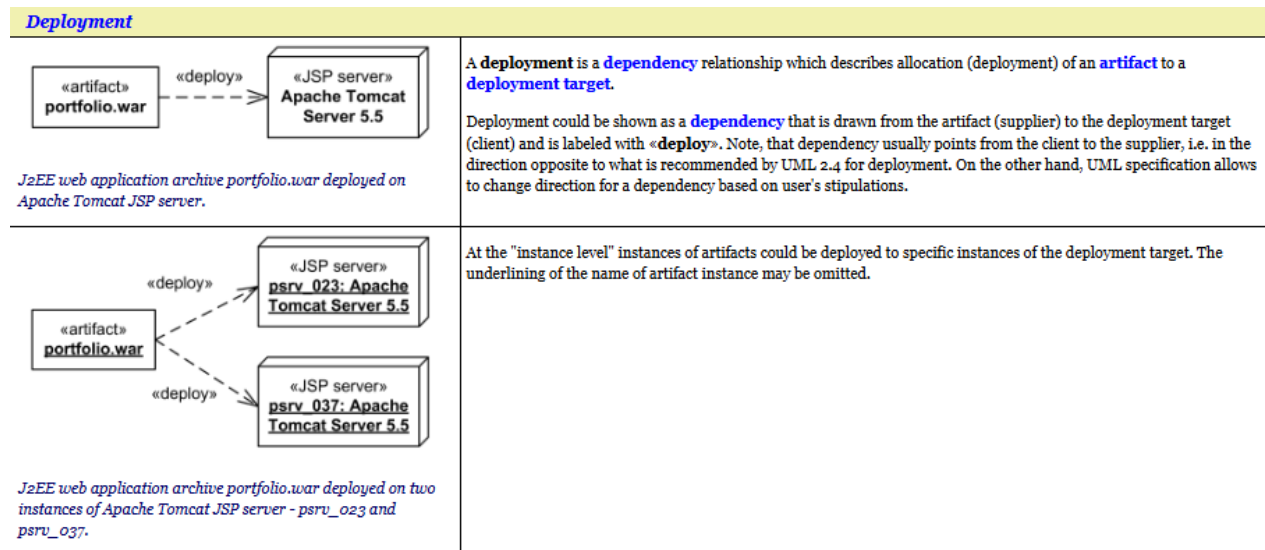


Figure 10: Deployment Diagram - Deployment [5]

2.5 Implementation Views

This view shows how the architecture is broken down into different layers. For this architecture, we show this using a layer diagram. Below, the notation used is shown, along with their definitions:

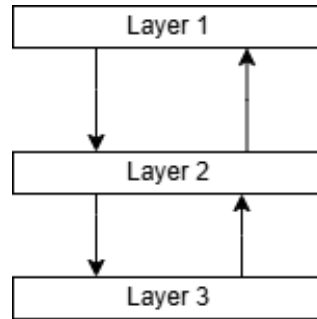


Figure 11: Layer Diagram [16]

In the diagram above, each rectangle represents a layer, the arrows point to the communication of the layers (the layers can use the layer it communicates to). In the diagram above, layer 1 can communicate to layer 2, layer 2 can communicate to layers 1 and 3, but layer 1 cannot communicate to layer 3 and vice versa [16].

With the implementation, we are also concerned with how the Knock Knock Protocol works, so a UML state machine diagram is used to show the transitions in the protocol's state. The notation is the same as the activity diagram, except the activity state represents the state of the system. We also support this view with a UML activity diagram to show the structure of how the Knock Knock Protocol is implemented.

3 Architectural Goals and Constraints

3.1 Architectural Goal

The goal of the architecture is to give a client (an employee) a knock-knock joke when they get bored in the office. The system does not use sensitive information, so we are not concerned with encrypting data in transit. The system is also command line based, so we are not worried about fast strobing lights affecting the safety of the employee. The architecture makes use of jar files, making the system very portable and easily distributable (see section 7 for how the architecture is deployed).

3.2 Architectural Constraints

The architecture is designed to be implemented with the Java language and can only be used by one employee at a time. Since it is written in Java, any conventional operating system such as Windows, Linux, Unix, and Mac can be used. Obscure operating systems such as TempleOS may not support the architecture. The hardware constraints of the architecture are listed in section 9. We do not include the number of jokes in this architecture, but at a minimum, there should be 1 joke, and no more than a few hundred jokes (of which may be overkill).

4 Use-Case View

The use case for this architecture is an employee getting the system to tell them a joke. This is shown below:

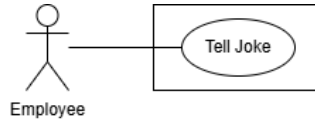


Figure 12: Use Case Diagram for Knock Knock

4.1 Use-Case Realizations

Use case (main flow of events):

1. The user is told “Knock Knock” by the server.
2. The user responds with a form of “who’s there?” (capitalization does not matter).
3. The server responds with the first half the the joke.
4. The user responds with the first half of the joke with a space and the word, in the form, “who”.
5. The server completes the joke and asks the user if they want to go again.

Alternative flow (invalid input):

1. When asked “Knock Knock”, if the input is not in the form of “who’s there?”, display a message to correct the user and restart the joke.
2. When prompted with a message after “who’s there?”, if the input is not the message repeated followed by a space and the word “who”, display a message to correct the user and restart the joke.

Alternative flow (yes or no):

1. If the user selects “y” to hear another joke, the server will start the process over with another joke.
2. If the user selects anything else to hear another joke, the program will terminate.

5 Logical View

This view shows how the architecture is setup as classes, the class diagram for the architecture is shown below:

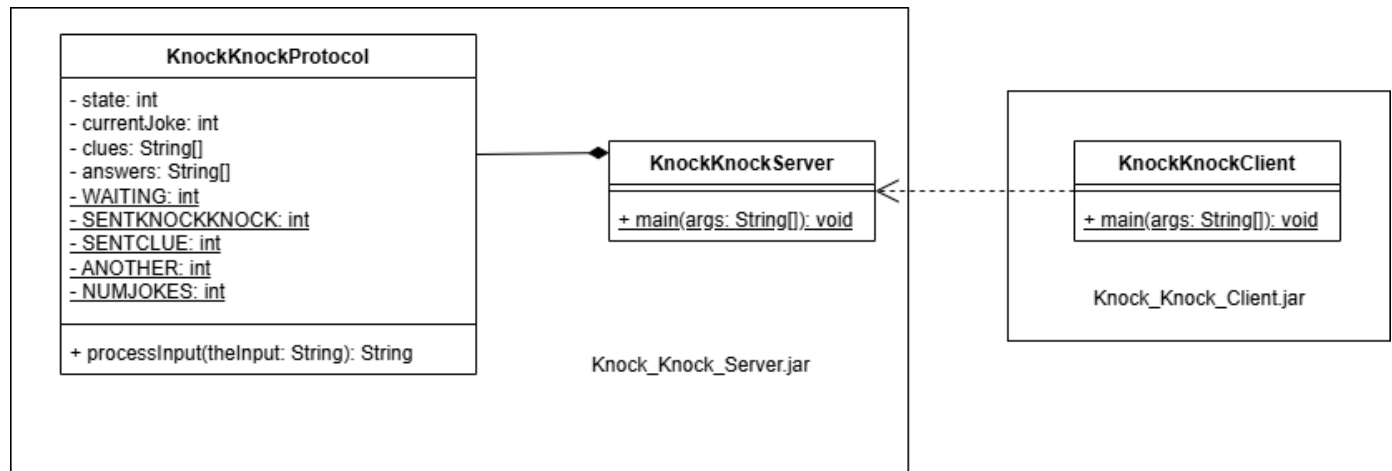


Figure 13: Class Diagram for Knock Knock

In the diagram, we specify the classes in each jar file, along with how the client depends on the server to run before it can execute the client program as intended.

6 Process View

This view shows how the architecture is broken down into processes, and how those processes communicate through each step of the architecture. The activity diagram for the Knock Knock Server and Client are shown below, with the details of the protocol omitted. The protocol itself contains one method (`processInput`), which is shown in more detail in section 8.

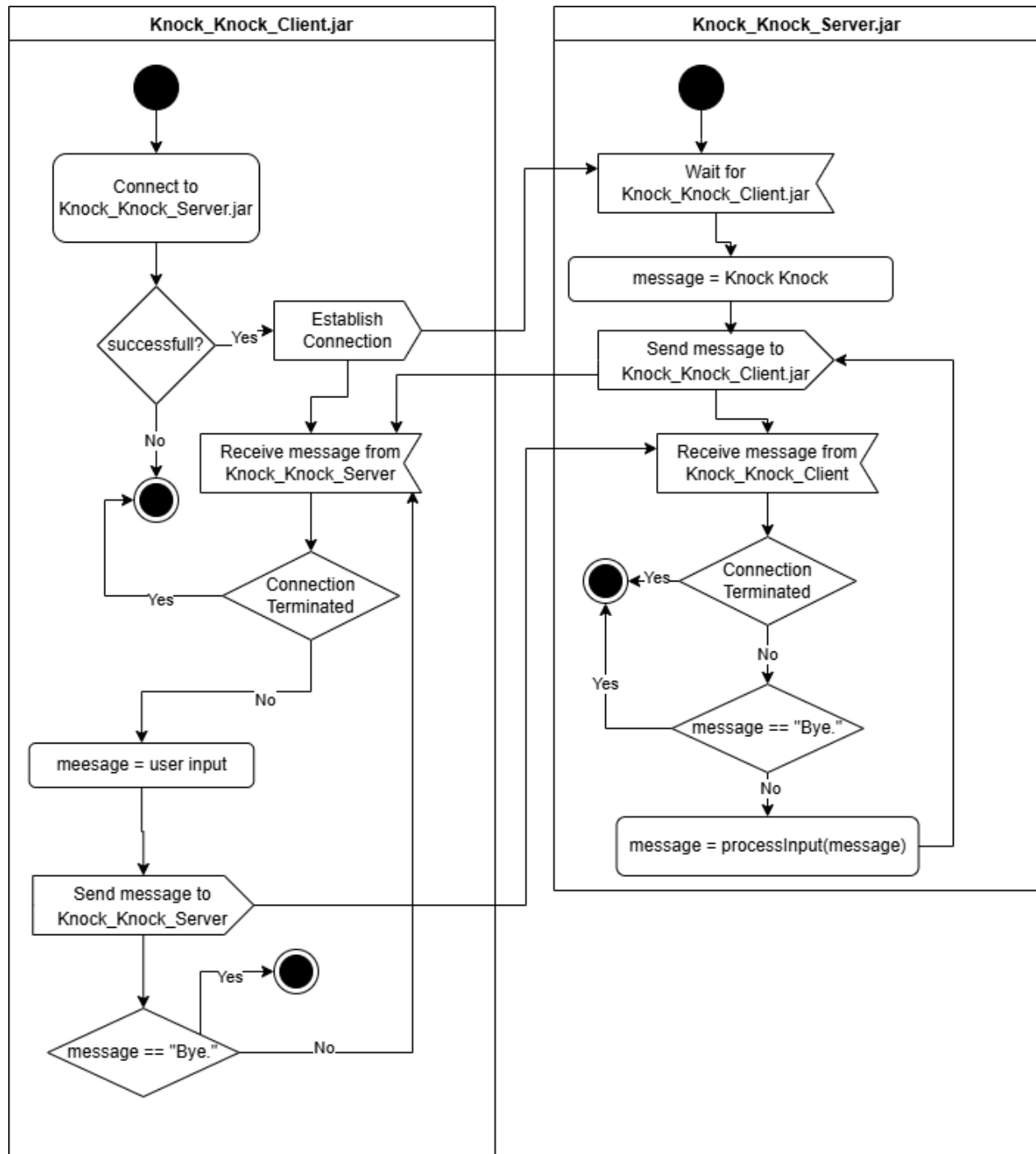


Figure 14: Activity Diagram for Knock Knock jar files

7 Deployment View

This view shows how the server and client are deployed in a corporate environment. The UML deployment diagram is shown below:

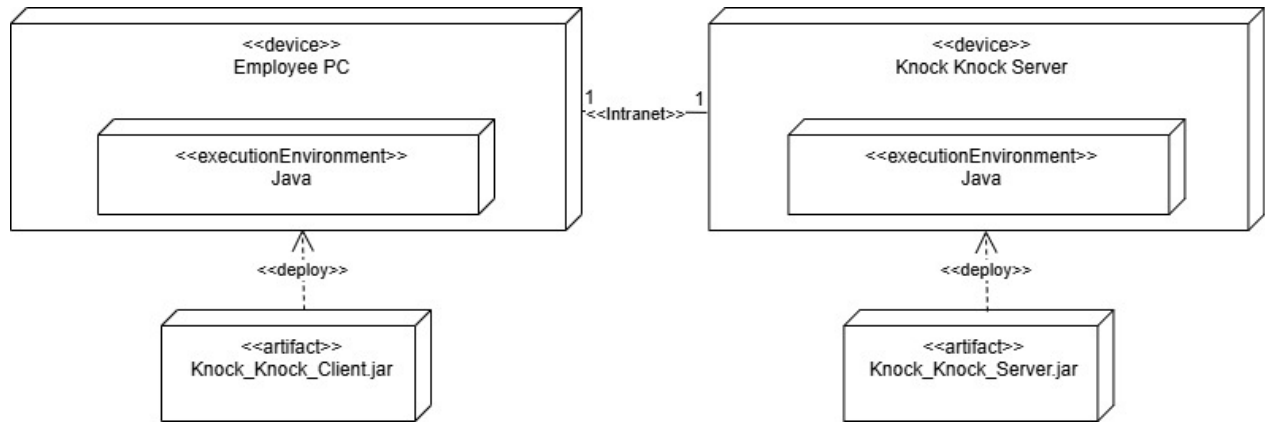


Figure 15: Deployment Diagram for Knock Knock

8 Implementation View

This view shows the architecture in terms of abstracted layers. The layers of the architecture are shown below:

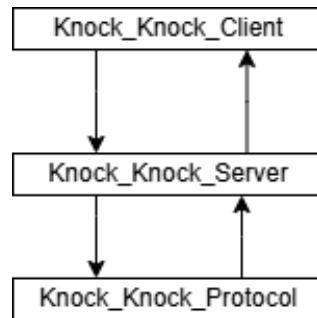


Figure 16: Layer Diagram for Knock Knock

8.1 Layers

The Knock_Knock_Server and Knock_Knock_Client layers are shown in detail with their respective .jar files in section 6, excluding the details of the protocol itself. The only method in the protocol is used in section 6 as processInput(message), but the details are omitted. Therefore, the state machine diagram of the protocol and its activity diagram are both shown below:

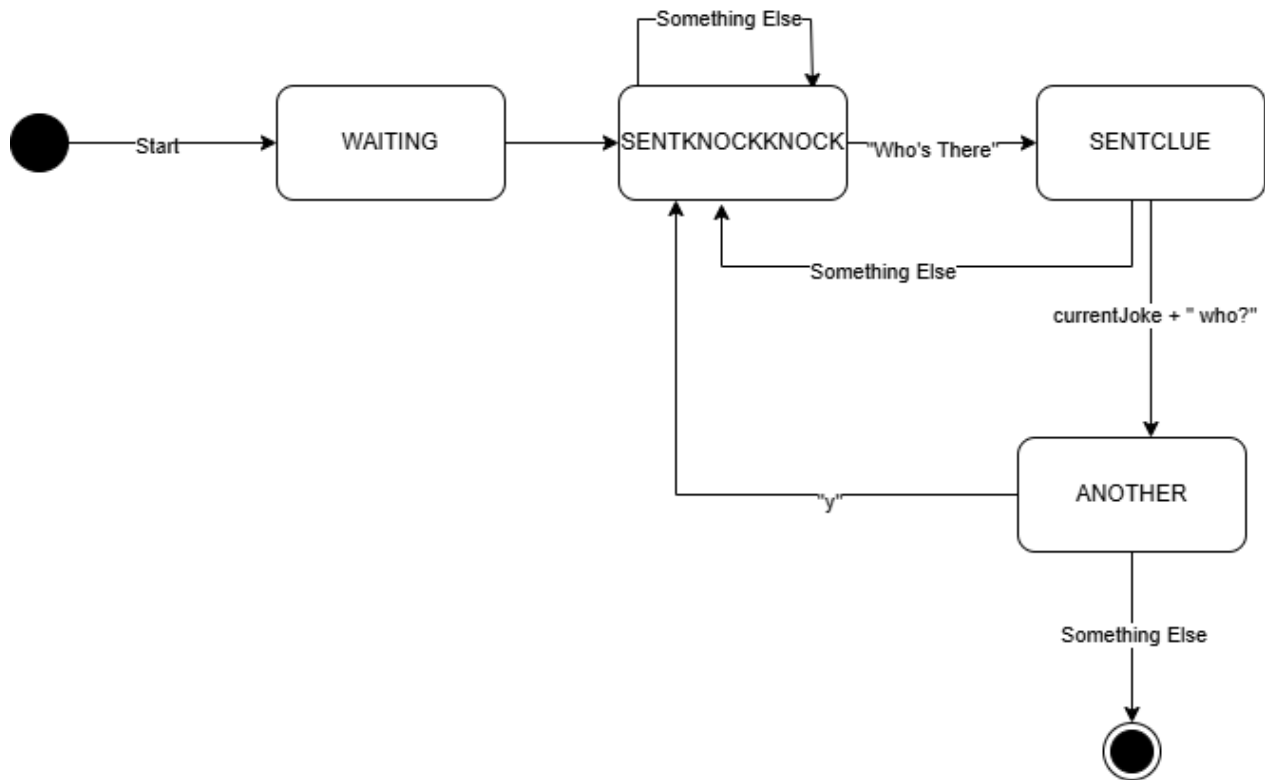


Figure 17: State Machine Diagram for the Knock Knock Protocol

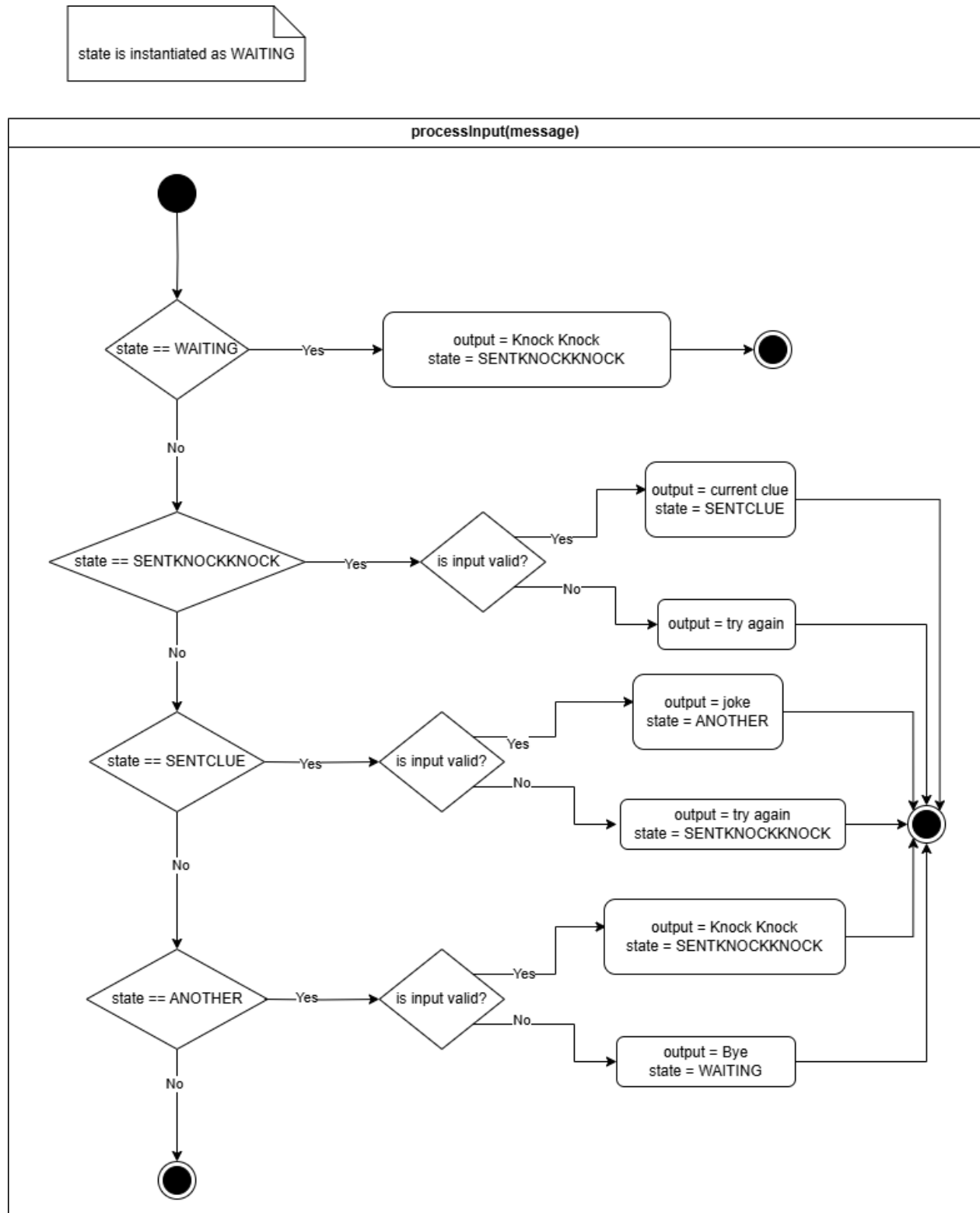


Figure 18: Activity Diagram for the Knock Knock Protocol

9 Size and Performance

So far in the architecture, we have omitted the details including the amount of jokes and clues. That detail can be left for implementing the architecture itself. The more jokes that are in the system, the more clues there are. Therefore, the defining characteristic of the size of the architecture goes into the number of jokes that are included.

The performance of the architecture is dependent of the local network speed and the processing speed of the server and client's device. The system does not require a lot of processing speed unless there are thousands, possibly millions, of jokes, which would be unnecessary as an employee would have time to go through thousands of jokes in a day. However, if the local network's speed is a few kilobits per second, there could be some performance issues. A large amount of words in a joke could also hinder performance if the network speed is slow.

With the target audience, there should not be a large amount of jokes and no more than a sentence per joke. Therefore, the architecture would be small and lightweight and can run on a minimal system, that is, a system with a low download/upload network speed, which should be no smaller than 0.5 megabits per second. Next, the hardware of the system has a minimum of one core on the CPU and enough storage for the operating system and the architecture itself. The architecture itself should not exceed 1 MB after implementation.

10 Quality

The architecture will be user-friendly and reliable. When there is not an available connection, it will clearly let the user know. Since the architecture makes use of jar files, the architecture is highly portable and can be moved at any point to another device.